Домашнее задание: защищенные маршруты и авторизация в React Router

Максимум 12 баллов

Цель

Закрепить знания о работе с защищенными маршрутами в React. Научиться реализовывать авторизацию с токеном, управлять доступом к страницам через `ProtectedRoute`, использовать `Outlet` и контекст авторизации. Попрактиковаться в построении полноценной структуры с защищенными и общедоступными зонами.

Структура проекта

```
src/
└── features/
    └── authRouting/
        ├── AuthContext.tsx
        ├── useAuth.ts
        ├── ProtectedRoute.tsx
        ├── LoginPage.tsx
        ├── ProfilePage.tsx
        ├── PublicPage.tsx
        └── AppRouter.tsx
```

Общие рекомендации

- Реализуйте все через `React Router v6+` с использованием `<Routes>` и `<Route>`.
- Используйте API `https://api.v2.react-learning.ru/api`.
- Структурируйте проект по фичам.
- Проверяйте, что редиректы работают и доступ к приватным маршрутам действительно ограничен.

Основная часть (до 9 баллов)

1. Компонент `ProtectedRoute` и `useAuth` (3 балла)

Создайте компонент `ProtectedRoute`, который проверяет авторизацию пользователя и перенаправляет на `/login`, если токена нет.
Информацию о пользователе храните в `AuthContext`, а доступ к ней — через хук `useAuth()`.

Требования

- Проверка доступа через контекст, а не напрямую через `localStorage`.
- Редирект на `/login` при отсутствии авторизации.
- Отображение вложенных маршрутов через `<Outlet />`.

Критерии

- Использован `useContext` + `Outlet` — 1 балл.
- Редирект через `<Navigate replace />` — 1 балл.
- Поведение корректно при обновлении страницы — 1 балл.

2. Страница `LoginPage` (3 балла)

Создайте страницу входа, которая использует API `POST /auth/login` для входа. Сохраняйте токен в `localStorage` и обновляйте контекст авторизации. После входа — редирект на `/profile`.

Требования

- Обязательные поля: email и password.
- Асинхронная отправка запроса.
- После успешного входа — переход на защищенную страницу.

Критерии

- Логин работает и сохраняет токен — 1 балл.
- Контекст обновляется корректно — 1 балл.
- После логина идет редирект — 1 балл.

3. Защищенная страница `ProfilePage` (3 балла)

Создайте компонент `ProfilePage`, доступный только авторизованным пользователям. На этой странице отобразите имя пользователя, полученное с помощью `GET /users/me`.

Требования

- Страница недоступна без логина.
- После авторизации отображается имя пользователя.
- При обновлении страницы данные пользователя восстанавливаются.

Критерии

- Проверка авторизации работает — 1 балл.
- Информация загружается с сервера — 1 балл.
- Поведение корректно после F5 — 1 балл.

Дополнительная задача (до 3 бонусных баллов)

Страница `PublicPage` + автоматический выход

Создайте открытый маршрут `/public`, доступный всем. Добавьте кнопку «Выйти» на странице профиля, которая удаляет токен, очищает контекст и возвращает пользователя на `/login`.

Рекомендации

- Используйте `navigate("/login", { replace: true })` при выходе.
- После выхода защищенные маршруты должны стать недоступны.

Критерии

- Кнопка выхода работает и очищает контекст — 1 балл.
- Происходит редирект на `/login` — 1 балл.
- После выхода доступ к `/profile` невозможен — 1 балл.